[TÊN TRƯỜNG / HỌC VIỆN]

[TÊN KHOA / VIỆN]

BÁO CÁO BÀI THỰC HÀNH 6

GIAO TIẾP PHI NGÔN NGỮ VÀ ĐIỀU KHIỂN TURTLEBOT 4 BẰNG CỬ CHỈ TAY

Non-verbal Human–Robot Interaction
Using Hand Gesture Recognition

Học phần:	[Tên học phần]
Giảng viên hướng dẫn:	[Họ và tên giảng viên]
Sinh viên thực hiện:	[Họ và tên sinh viên]
Mã số sinh viên:	[Mã số sinh viên]
Lớp / Nhóm:	[Thông tin lớp / nhóm]

[Địa điểm], tháng [. . .] năm 2026

Mục lục

1 Giới thiệu bài toán **1**
1.1 Mục tiêu và phạm vi . . . 1

2 Cấu trúc và kiến trúc hệ thống **1**
2.1 Sơ đồ kiến trúc tổng thể . . . 1
2.2 Cây thư mục và chức năng từng file . . . 1
2.3 Node, topic và dữ liệu trao đổi . . . 2

3 Nhận diện cử chỉ tay **2**
3.1 Luồng xử lý ảnh . . . 2
3.2 Phân biệt Wave với Open Palm . . . 3

4 Điều khiển hành vi robot **4**
4.1 Bảng ánh xạ cử chỉ . . . 4
4.2 Máy trạng thái chọn–xác nhận . . . 4
4.3 Quỹ đạo và phản hồi odometry . . . 5
4.4 Điều kiện chạy và xử lý dừng . . . 5

5 Triển khai và vận hành **5**
5.1 Môi trường thực hiện . . . 5
5.2 Câu lệnh khởi chạy . . . 5
5.3 Kiểm tra trước demo và khôi phục GUI . . . 6

6 Kết quả thực nghiệm và đánh giá **7**
6.1 Dữ liệu và kịch bản . . . 7
6.2 Minh họa sáu cử chỉ . . . 7
6.3 Kết quả chuyển động và xác nhận . . . 8
6.4 Dừng, hủy và mất tín hiệu . . . 9
6.5 Vấn đề thực tế và cách xử lý . . . 9
6.6 Giới hạn kết luận . . . 9

7 Kết luận và hướng phát triển **10**
7.1 Kết quả đạt được . . . 10
7.2 Hướng cải tiến . . . 10

Tài liệu tham khảo **10**

Phụ lục: Bộ minh chứng thực nghiệm **11**

Chương 1. Giới thiệu bài toán

1.1. Mục tiêu và phạm vi

Bài thực hành xây dựng chuỗi **webcam → nhận diện cử chỉ → ROS 2 → TurtleBot 4 trong Gazebo**. Hai node ứng dụng sử dụng MediaPipe để nhận năm cử chỉ tĩnh, bổ sung Wave theo chuyển động và điều khiển ba hành vi: chào, bốn chu kỳ tiến–lùi, một vòng tròn. Robot cùng controller nền tảng được tái sử dụng từ các package chính thức [6, 8].

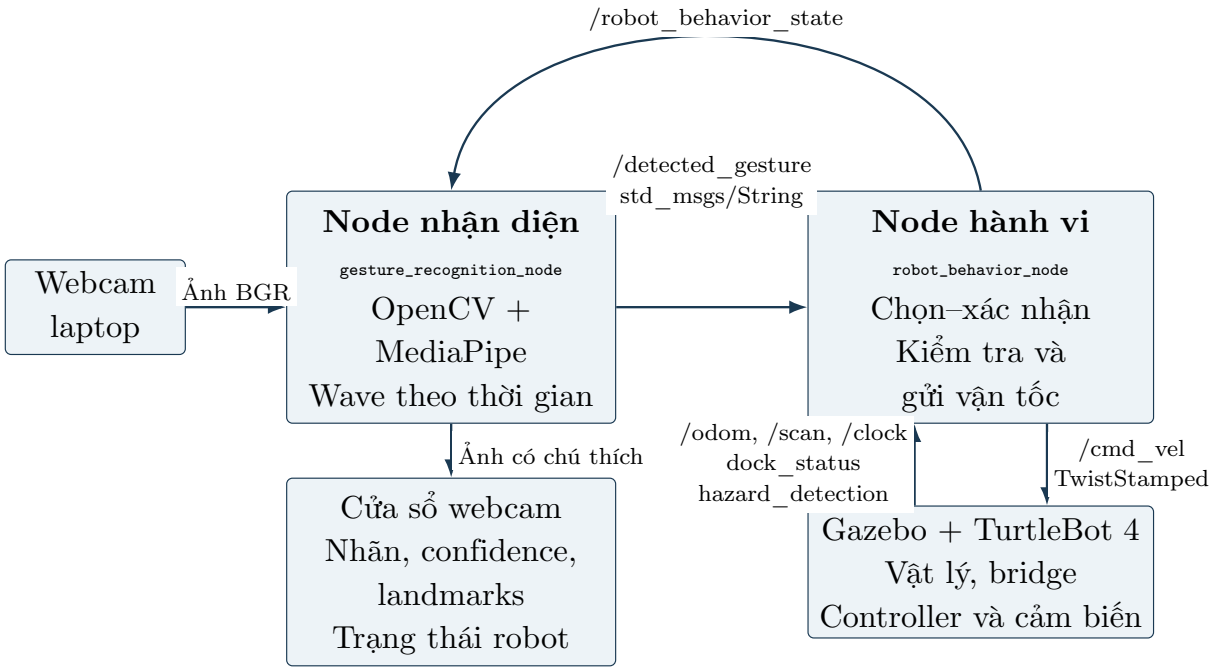
- **Chọn–xác nhận:** chọn hành vi chưa làm robot chạy; cần Thumb Up hợp lệ.
- **Dừng/hủy:** Open Palm dừng, Closed Fist hủy, mất tín hiệu làm robot dừng.
- **Minh chứng:** video webcam thật ngày 24/10/2026, ảnh và odometry; không dùng teleop hoặc cử chỉ giả để lái robot trong demo.

Lượt thực nghiệm gồm một người dùng và một video 180,05 giây. Báo cáo đánh giá chức năng của lượt chạy, chưa suy ra độ chính xác trên nhiều người hoặc điều kiện ánh sáng.

Chương 2. Cấu trúc và kiến trúc hệ thống

2.1. Sơ đồ kiến trúc tổng thể

Hình 1 phân biệt hai node chính, đường lệnh và đường phản hồi. OpenCV, MediaPipe, WaveDetector là các thành phần bên trong node nhận diện. Khối Gazebo gộp robot, bridge và controller có sẵn.



Hình 1: Kiến trúc hệ thống: nhận diện, quyết định hành vi và phản hồi mô phỏng.

2.2. Cây thư mục và chức năng từng file

Cây ở Hình 2 đặt chú thích ngay cạnh file. Package Bài 6 tách riêng với bài SLAM/Nav2/YOLO trước; chỉ tái sử dụng tiện ích môi trường và launch nền tảng.

CÂY MÃ NGUỒN	CHỨC NĂNG
src/turtlebot4_gesture_control/	Package Bài 6; tách khỏi các bài trước.
turtlebot4_gesture_control/	Các mô-đun Python.
__init__.py	Khai báo package Python.
gesture_recognition_node.py	Webcam, MediaPipe, nhãn và ảnh.
gesture_logic.py	Wave và máy trạng thái hành vi.
robot_behavior_node.py	Cảm biến ROS và lệnh vận tốc.
config/	Thông số vận hành.
behavior.yaml	Tốc độ, quãng đường và timeout.
launch/	Các lệnh khởi chạy.
arena.launch.py	Mô phỏng; robot giữa sân, dock ở góc.
behavior.launch.py	Nạp YAML và chạy node hành vi.
gui.launch.py	Mở lại GUI, giữ server đang chạy.
worlds/	Tài nguyên môi trường.
gesture_arena.sdf	Sân 16 x 16 m, sàn và tường bao.
scripts/	Chuẩn bị phụ thuộc.
setup_gesture.sh	Tạo venv và tải model MediaPipe.
test/	Kiểm thử logic và launch.
test_gesture_logic.py	Xác nhận, dừng, Wave, tiến độ.
test_arena_world.py	World bridge và vị trí dock/TF.
resource/	Đăng ký package với ament.
setup.py	Executable và tài nguyên cài đặt.
setup.cfg	Vị trí cài các executable Python.
package.xml	Tên package, phụ thuộc, kiểu build.
README.md	Câu lệnh và quy trình demo.
VALIDATION.md	Kết quả và giới hạn kiểm thử.

Hình 2: Cây thư mục package với chức năng ngắn gọn đặt ngay cạnh từng file.

2.3. Node, topic và dữ liệu trao đổi

ROS 2 hỗ trợ một node đồng thời publish và subscribe nhiều topic [4]. Vai trò hai node ứng dụng được tóm tắt như sau:

Node	Đầu vào	Xử lý / đầu ra
`gesture_recognition_node`	Webcam; trạng thái hành vi.	Nhận diện, lọc, hiển thị; publish nhãn, ảnh và debug.
`robot_behavior_node`	Cử chỉ; odom, scan, clock, dock, bumper.	Quản lý trạng thái; publish vận tốc và trạng thái.

Bảng 1: Hai node chính trong chuỗi điều khiển.

Trong lượt quay còn có node quan sát `lab06_demo_observer` để lưu sự kiện/odom và nhắc bước. Node này chỉ subscribe, không gửi cử chỉ hay vận tốc.

`/cmd_vel` dùng `TwistStamped`, gồm header và twist [5]; lệnh sử dụng `linear.x`, `angular.z`, timestamp mô phỏng và frame `base_link`. Trạng thái có QoS transient local; các cảm biến dùng QoS sensor data.

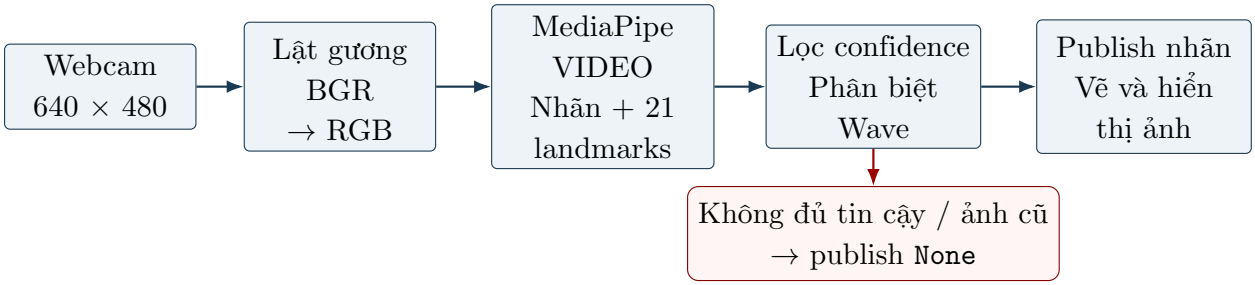
Chương 3. Nhận diện cử chỉ tay

3.1. Luồng xử lý ảnh

OpenCV đọc webcam bằng `VideoCapture` [3]. Model `gesture_recognizer.task` chạy chế độ `VIDEO`, một bàn tay, timestamp tăng nghiêm ngặt và lời gọi `recognize_for_video` [2].

Topic	Kiểu message	Vai trò
/detected_gesture	std_msgs/msg/String	Nhãn hoặc None.
/cmd_vel	geometry_msgs/msg/TwistStamped	Vận tốc và timestamp.
/odom	nav_msgs/msg/Odometry	Tiến độ vị trí/góc.
/scan	sensor_msgs/msg/LaserScan	Khoảng đo vật cản.
/clock	rosgraph_msgs/msg/Clock	Đồng hồ mô phỏng.
/dock_status	irobot_create_msgs/msg/DockStatus	Robot có ở dock hay không.
/hazard_detection	irobot_create_msgs/msg/HazardDetectionVector	Phản hồi bumper.
/robot_behavior_state	std_msgs/msg/String	JSON trạng thái và ready.
/gesture_recognition/image	sensor_msgs/msg/Image	Ảnh đã chú thích.
/gesture_recognition/debug	std_msgs/msg/String	Nhãn gốc, confidence, thời gian xử lý.

Bảng 2: Giao diện topic của bản triển khai ROS 2 Jazzy.



Hình 3: Luồng nhận diện. Kết quả không hợp lệ không được giữ làm lệnh cũ.

Capture/suy luận nằm ở một luồng riêng; timer ROS chỉ lấy kết quả gần nhất để tránh hàng đợi ảnh cũ.

Điều kiện	Cấu hình	Tác dụng
Confidence phân loại	$\geq 0{,}65$	Loại kết quả không đủ tin cậy.
Tuổi ảnh tối đa	0,35 s	Ảnh cũ tạo None.
Publish kết quả	10 Hz	Nhịp gửi tín hiệu điều khiển.
Giới hạn capture	15 fps	Giới hạn vòng đọc/xử lý, không phải FPS đã đo.

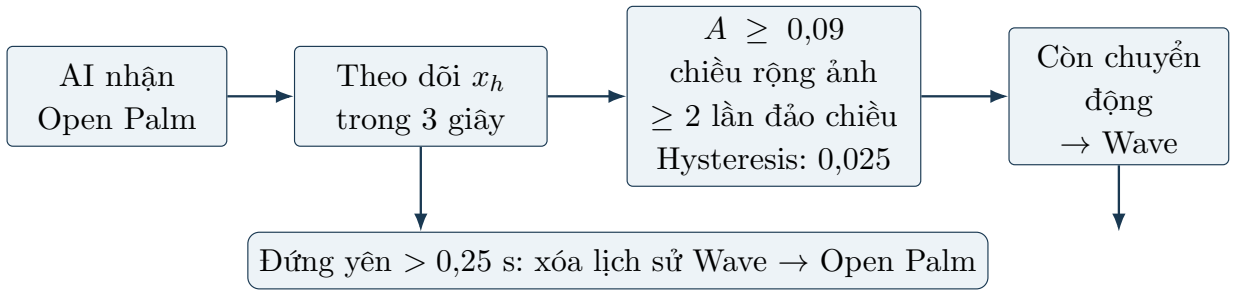
Bảng 3: Các điều kiện xử lý ở node nhận diện.

Landmarks còn xuất hiện chưa chắc nhãn đã hợp lệ. Năm nhãn tĩnh dùng trong bài là Pointing Up, Victory, Thumb Up, Open Palm và Closed Fist [1]; các nhãn khác hoặc không đủ điều kiện đều tạo None.

3.2. Phân biệt Wave với Open Palm

Wave không phải nhãn dựng sẵn của model. Hệ thống kết hợp Open Palm với chuyển động trung bình của bốn đầu ngón:

$$x_h = \frac{x_8 + x_{12} + x_{16} + x_{20}}{4}, \qquad A = \max(x_h) - \min(x_h). \tag{1}$$



Hình 4: Wave là cử chỉ động; bàn tay mở đứng yên vẫn là lệnh dừng.

Một lần quét ngang chưa đủ thành Wave. Lịch sử được làm mới khi đổi tay, mất mẫu lâu hoặc vị trí nhảy bất thường; một frame không chắc chắn vẫn publish None. Điểm hiển thị của Wave là confidence Open Palm kết hợp điều kiện chuyển động, không phải điểm của một mô hình Wave huấn luyện riêng.

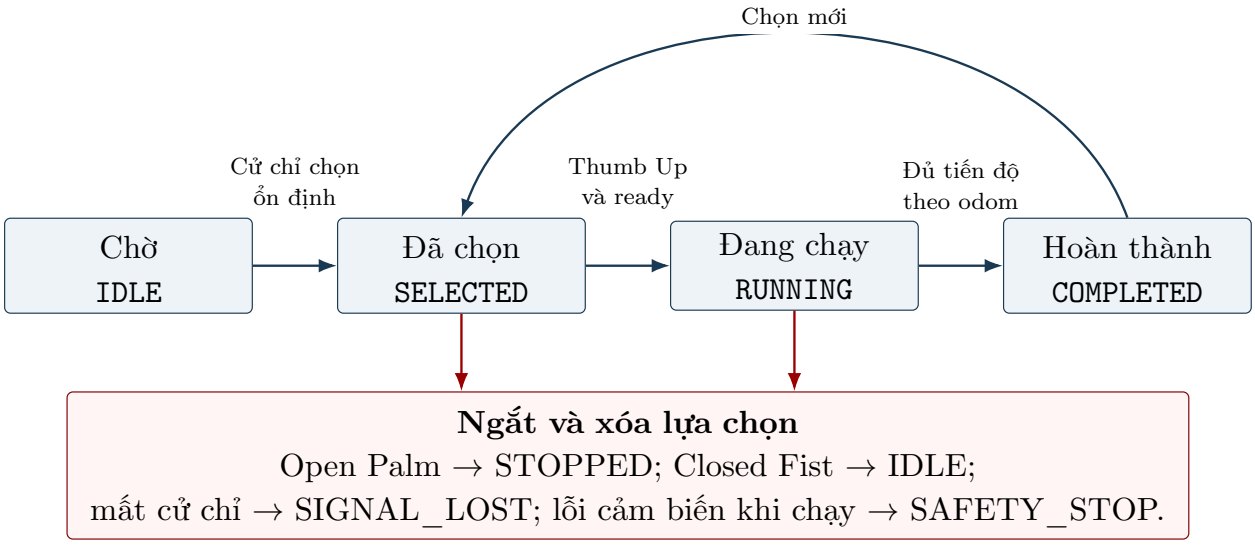
Chương 4. Điều khiển hành vi robot

4.1. Bảng ánh xạ cử chỉ

Cử chỉ	Hành vi	Điều kiện
Wave	Chọn quay chào trái–phải.	Chờ Thumb Up.
Pointing Up	Chọn bốn chu kỳ tiến–lùi.	Chờ Thumb Up.
Victory	Chọn đi một vòng tròn.	Chờ Thumb Up.
Thumb Up	Xác nhận hành vi đã chọn.	Robot đủ điều kiện chạy.
Open Palm	Dừng, xóa lựa chọn.	Không chờ xác nhận.
Closed Fist	Hủy, trở về chờ.	Không chờ xác nhận.

Bảng 4: Ánh xạ sáu cử chỉ với hành vi TurtleBot 4.

4.2. Máy trạng thái chọn–xác nhận



Hình 5: Luồng trạng thái chính. Chọn/xác nhận bị lọc theo thời gian; dừng/hủy được ưu tiên.

- Chọn và xác nhận phải ổn định 0,35 s; lựa chọn hết hạn sau 10 s.

- Giữ một cử chỉ không phát lệnh lặp liên tục. Trong RUNNING, cử chỉ chọn mới không thay hành vi hiện tại.
- Hoàn thành hoặc bị ngắt đều xóa lựa chọn. Xác nhận khi thiếu dữ liệu tạo NOT_READY.
- Vận hành bằng cách giữ Thumb Up trong khung hình; heartbeat từ cử chỉ hợp lệ duy trì điều khiển, nhưng Open Palm/Closed Fist luôn dừng/hủy.

4.3. Quỹ đạo và phản hồi odometry

Hành vi	Lệnh và chuỗi pha	Điều kiện hoàn thành
Tiến–lùi	Tám lượt luân phiên; mỗi lượt 0,35 m; $v = \pm 0{,}24$ m/s.	Mỗi pha đạt tổng dịch chuyển mục tiêu.
Vòng tròn	$R = 0{,}60$ m; $v = 0{,}24$ m/s; $\omega = v/R = 0{,}40$ rad/s.	$\sum \lvert\Delta\theta\rvert \geq 2\pi$.
Chào	Góc pha +0,45, −0,90, +0,45 rad; $\lvert\omega\rvert = 0{,}90$ rad/s.	Đạt tiến độ từng pha quay.

Bảng 5: Ba chuyển động được thực hiện theo phản hồi, không chỉ bằng thời gian chờ cố định.

Tiến độ dùng $\Delta s = \sqrt{\Delta x^2 + \Delta y^2}$ hoặc biến thiên yaw đã chuẩn hóa. Controller còn tăng/giảm tốc nên vị trí thực có thể vượt ngưỡng sau khi chuyển pha; triển khai hiện tại chưa điều khiển vị trí theo sai số có dấu.

4.4. Điều kiện chạy và xử lý dừng

Điều kiện	Ngưỡng / trạng thái	Xử lý
Cử chỉ chọn/xác nhận	Ổn định 0,35 s	Chấp nhận một sự kiện.
Thiếu cử chỉ hợp lệ	Quá 0,65 s	SIGNAL_LOST, vận tốc về 0.
Odom/scan không còn mới	Quá 0,80 s	Không cho bắt đầu; đang chạy thì dừng.
Robot docked / có bumper	ready không đạt	Chặn chạy hoặc SAFETY_STOP.
Khoảng đo LiDAR nhỏ	Dưới 0,35 m theo hướng chạy	OBSTACLE_STOP.
Hành vi quá lâu	Quá 180 s thực	TIMEOUT.

Bảng 6: Các lớp bảo vệ trong điều khiển.

Timer hành vi 20 Hz và watchdog dùng đồng hồ steady; timestamp lệnh dùng thời gian mô phỏng. Ready còn yêu cầu scan hợp lệ trước/sau và clock đã bắt đầu. Kiểm tra LiDAR là điều kiện dừng, không phải bộ lập kế hoạch tránh vật cản; sân demo được để trống, dock đặt ở góc $(6, 6)$.

Chương 5. Triển khai và vận hành

5.1. Môi trường thực hiện

5.2. Câu lệnh khởi chạy

Nạp môi trường dưới đây trong **mỗi terminal**; build và cài model chỉ cần thực hiện khi chuẩn bị dự án:

```
cd ~/tb4_ws
```

Thành phần	Cấu hình
Nền tảng	Ubuntu 24.04, ROS 2 Jazzy, Python 3.12.
Mô phỏng	Gazebo Harmonic, TurtleBot 4 Standard, sân 16 × 16 m.
Nhận diện	MediaPipe 0.10.21, OpenCV 4.6.0, NumPy 1.26.4; venv riêng.
Giao tiếp	Cùng ROS domain 76 và Gazebo partition tương ứng.
Ghi hình	MP4 H.264, 1920 × 1080, 20 fps; không âm thanh.

Bảng 7: Môi trường dùng trong lượt thực nghiệm.

```
export TB4_LAB_DOMAIN_ID=76
source src/turtlebot4_slam_nav/scripts/env.sh
export GZ_IP=127.0.0.1

# Build/setup when preparing the project
colcon build --symlink-install \
  --packages-up-to turtlebot4_gesture_control
source install/local_setup.bash
bash src/turtlebot4_gesture_control/scripts/setup_gesture.sh
```

Listing 1: Ba lệnh vận hành chạy trong ba terminal đã nạp môi trường.

```
# Terminal A: Gazebo
ros2 launch turtlebot4_gesture_control arena.launch.py \
  gui_render_engine:=ogre

# Terminal B: behavior
ros2 launch turtlebot4_gesture_control behavior.launch.py

# Terminal C: perception
./.venv-gesture/bin/python -m \
  turtlebot4_gesture_control.gesture_recognition_node \
  --ros-args -p camera_index:=0 -p confidence:=0.65 \
  -p max_fps:=15.0
```

Chờ `ready=True`, chọn cử chỉ rồi Thumb Up xác nhận; giữ tay được nhận diện khi robot chạy. Sửa YAML cần chạy lại node hành vi. Robot xuất phát ngoài dock trong sân mặc định nên không cần Undock.

5.3. Kiểm tra trước demo và khôi phục GUI

```
ros2 topic type /cmd_vel
ros2 topic hz /scan
ros2 topic hz /odom
ros2 topic echo /detected_gesture
ros2 topic echo /robot_behavior_state \
  --qos-durability transient_local
```

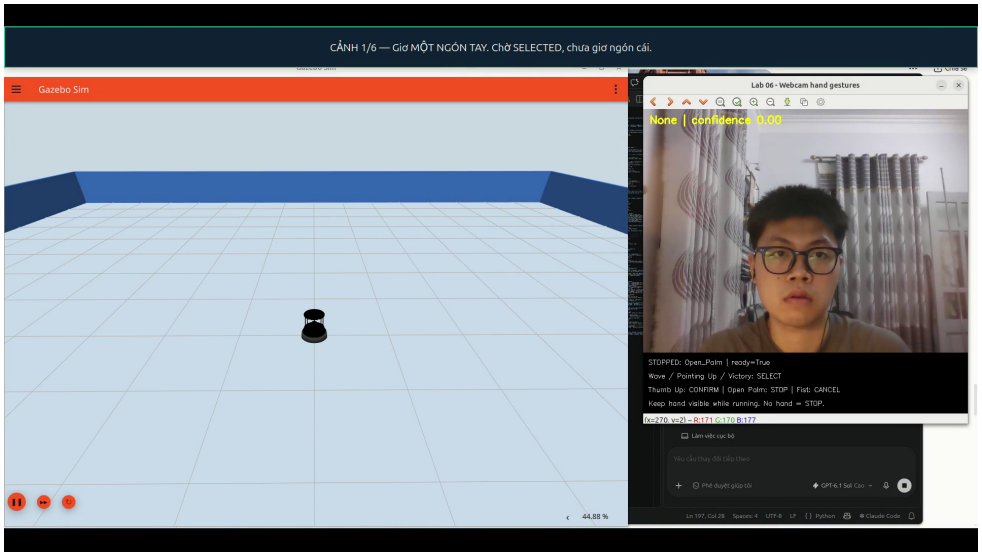
Các lệnh theo dõi chạy riêng, dừng bằng Ctrl+C khi chuyển lệnh. GUI dùng Ogre1, server/LiDAR dùng Ogre2. Mở lại GUI trong terminal cùng domain bằng lệnh sau; nếu server đã dừng, chạy lại arena.

```
ros2 launch turtlebot4_gesture_control gui.launch.py
```

Chương 6. Kết quả thực nghiệm và đánh giá

6.1. Dữ liệu và kịch bản

Bộ minh chứng [9] gồm video 180,05 giây ngày 24/10/2026, ảnh, log trạng thái và odometry. Người dùng làm cử chỉ thật; công cụ ghi chỉ quan sát topic. Thứ tự cảnh: tiến–lùi, vòng tròn, chào, dừng bằng Open Palm, hủy bằng Closed Fist và mất tín hiệu.



Hình 6: Bố trí ghi hình đồng thời Gazebo, webcam và thanh hướng dẫn.

6.2. Minh họa sáu cử chỉ



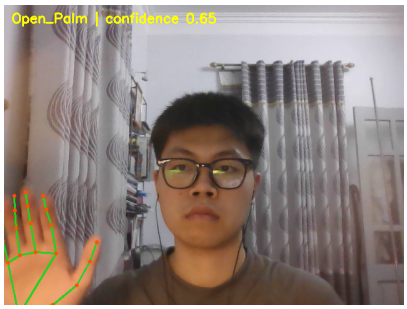
(a) Pointing Up



(b) Victory



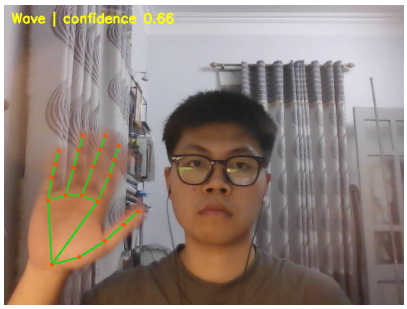
(c) Thumb Up



(d) Open Palm



(e) Closed Fist



(f) Wave

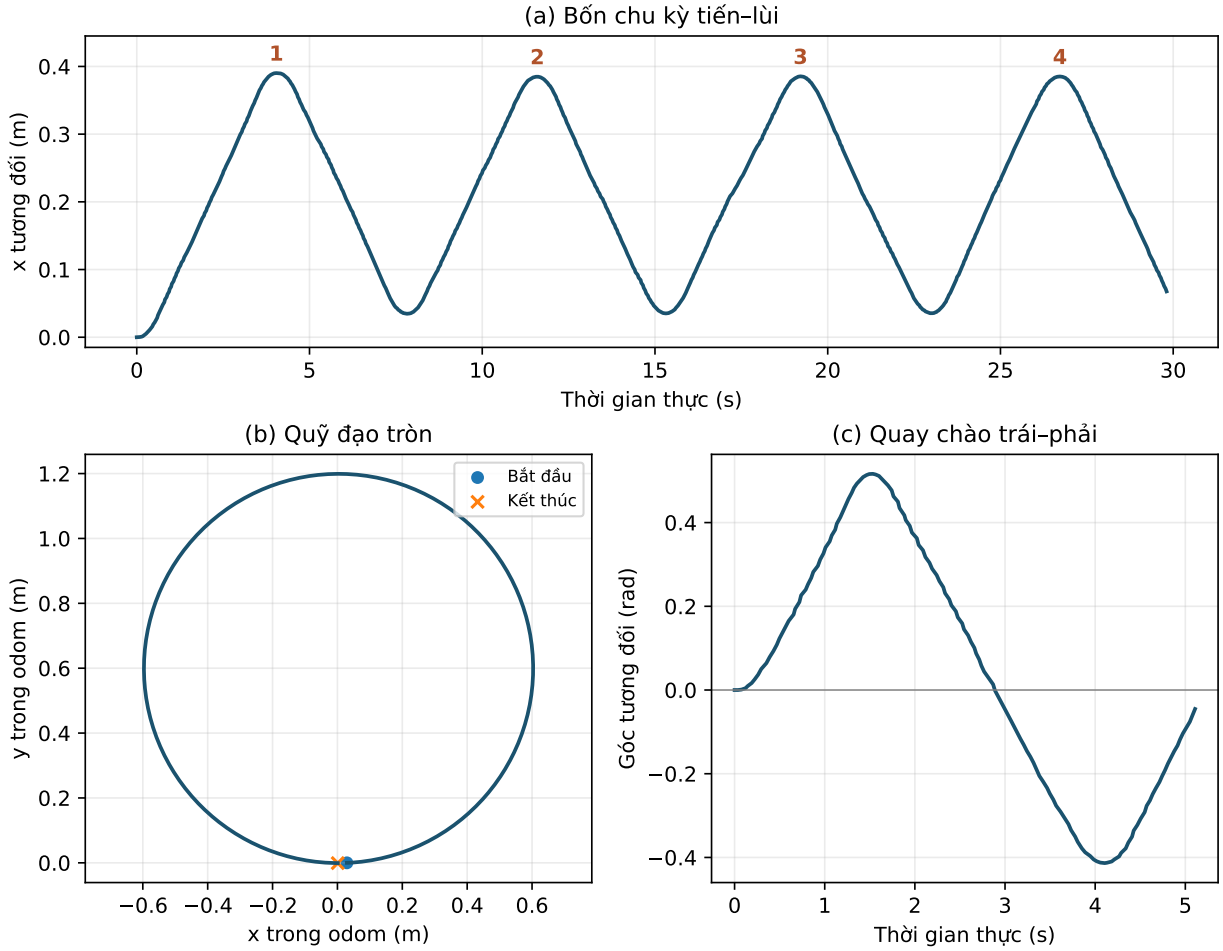
Hình 7: Ảnh thật trong lượt quay; chỉ lược phần hướng dẫn phía dưới khi dàn trang. Một ảnh Wave minh họa nhãn, còn điều kiện chuyển động được đối chiếu bằng video.

Nhãn	Confidence mẫu	Lưu ý
Pointing Up	0,7967	Mẫu một ngón.
Victory	0,8582	Mẫu hai ngón.
Thumb Up	0,6525	Gần ngưỡng 0,65.
Open Palm	0,6534	Tay gần biên ảnh.
Closed Fist	0,9742	Mẫu nắm tay.
Wave	0,6595	Điểm Open Palm cộng điều kiện động.

Bảng 8: Điểm của ảnh mẫu đầu tiên nhận được ở mỗi nhãn, không phải điểm trung bình hay tỷ lệ chính xác.

6.3. Kết quả chuyển động và xác nhận

Ở cảnh đầu, robot vẫn đứng yên khi Pointing Up đã tạo SELECTED; chỉ sau Thumb Up mới chuyển RUNNING. Log ghi đủ tám pha tiến/lùi. Hình 8 gộp dữ liệu của ba hành vi thành công để đối chiếu số chu kỳ, hình dạng quỹ đạo và thứ tự quay.



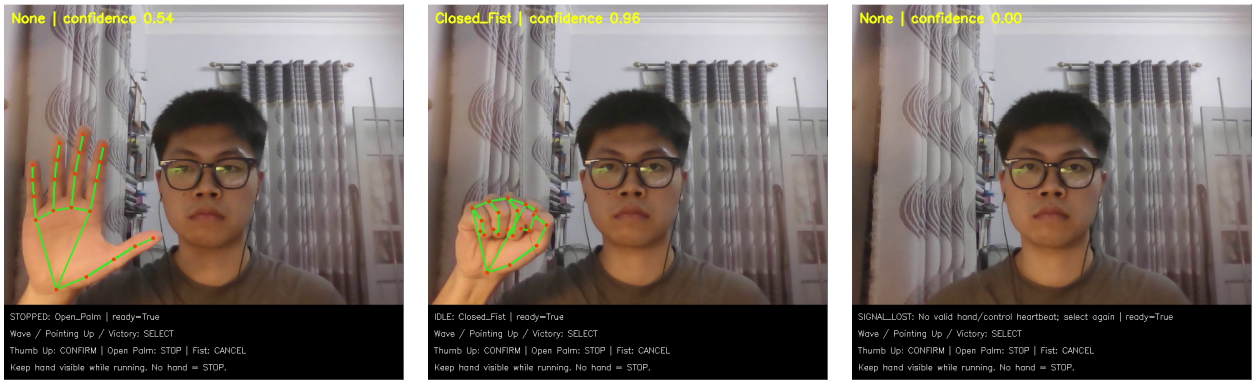
Hình 8: Tổng quan chuyển động từ odometry thật: bốn chu kỳ tiến–lùi, một vòng tròn và ba pha chào. Trục thời gian là đồng hồ thực.

Độ vượt ở pha tiến/quay phản ánh giới hạn của điều khiển tiến độ hiện tại: chưa dùng sai số vị trí có dấu và giảm tốc theo khoảng còn lại. Thời gian thực cũng không so trực tiếp với thời gian lý tưởng mô phỏng vì còn tăng/giảm tốc và real-time factor.

Hành vi	Thời gian thực	Quan sát đo được	Nhận xét
Tiến–lùi	29,85 s	Bốn đỉnh; tám pha; đỉnh đầu khoảng 0,390 m.	Có vượt ngưỡng pha 0,35 m.
Vòng tròn	35,70 s	Bán kính khớp khoảng 0,600 m; độ khép kín khoảng 0,028 m.	Đánh giá trong odom, chưa so với ground truth.
Chào	5,15 s	Góc tương đối từ khoảng $+0,516$ đến $-0,414$ rad; cuối $-0,045$ rad.	Đúng thứ tự, còn lệch góc.

Bảng 9: Phân tích ngắn theo dữ liệu của từng hành vi, không suy từ ảnh đứng.

6.4. Dừng, hủy và mất tín hiệu



(a) STOPPED (b) IDLE (c) SIGNAL_LOST

Hình 9: Cắt đúng vùng webcam/trạng thái từ ảnh ghi màn hình. Trong ảnh STOPPED, nhãn hiện tại đã dao động về None; lý do dừng lưu lại là Open Palm.

Cảnh	Nội dung	Trạng thái đạt	Mốc đạt (s)	Thử lại
1	Bốn chu kỳ tiến–lùi	`COMPLETED`	47,22	0
2	Một vòng tròn	`COMPLETED`	92,87	0
3	Chào sau Wave	`COMPLETED`	115,22	1
4	Open Palm khi đang chạy	`STOPPED`	144,57	1
5	Closed Fist khi đang chạy	`IDLE`	157,68	0
6	Đưa tay khỏi webcam	`SIGNAL_LOST`	171,52	0

Bảng 10: Sáu cảnh đạt sau thao tác; mốc đạt xấp xỉ trong lượt ghi. Thử lại là số đoạn gián đoạn, không phải số lỗi phân loại theo frame.

6.5. Vấn đề thực tế và cách xử lý

Ở cảnh 3, một lần lựa chọn Wave bị xóa khi tay trở về Open Palm trước xác nhận. Ở cảnh 4, một lần dừng do SIGNAL_LOST xảy ra trước khi nhận bàn tay mở. Video giữ cả hai đoạn; sau đó các cảnh được làm lại và đạt trạng thái đích.

6.6. Giới hạn kết luận

- Một người, một video chưa đủ để tính accuracy hay ma trận nhầm lẫn. Confidence mẫu không phải tỷ lệ đúng của hệ thống.

Hiện tượng	Nguyên nhân / quan sát	Cách xử lý
Vẫy vẫn ra Open Palm	Cấu hình cũ đòi ba đảo chiều/1,8 s; đặc trưng gần cổ tay dịch chuyển ít.	Theo dõi đầu ngón; hai đảo chiều/3 s; xóa lịch sử khi đứng yên.
Mất tín hiệu khi đổi tư thế	Khoảng None kéo dài quá 0,65 s; đã xảy ra ở cảnh 4.	Làm lại, mở tay dứt khoát trong khung; giữ nguyên cơ chế dừng.
NOT_READY	Gazebo domain 74, các node domain 76; thiếu sensor ở controller.	Đồng bộ ba terminal về domain 76; kiểm tra topic.
GUI thoát mã 139	Log Qt/Ogre2 có lỗi render; chưa xác định chính xác lỗi C++.	GUI Ogre1, server/sensor Ogre2; có launch mở lại GUI [7].

Bảng 11: Tổng hợp sự cố, bằng chứng và biện pháp đã áp dụng.

- 18 test tự động kiểm chứng logic xác nhận, chống lặp, dừng, Wave, tiến độ và launch; chúng không thay thế đánh giá nhận diện bằng tay thật.
- Video 20 fps, publish 10 Hz, giới hạn capture 15 fps là các đại lượng khác nhau. 884 ảnh preview không phải tổng frame video hoặc lượt suy luận.
- Chưa đo độ trễ dừng đầu–cuối, chưa so odom với ground truth và chưa thử vật cản động. Video chưa có cảnh riêng giơ Thumb Up sau dừng/mất tín hiệu để kiểm tra không tự chạy lại.

Chương 7. Kết luận và hướng phát triển

7.1. Kết quả đạt được

Hệ thống đã nhận sáu cử chỉ, giao tiếp qua ROS 2, chỉ thực hiện hành vi sau xác nhận và minh chứng ba chuyển động cùng dừng/hủy/mất tín hiệu. Các sơ đồ mô tả đúng luồng triển khai; ảnh và odometry của lượt quay là cơ sở đánh giá, gồm cả hai đoạn phải thao tác lại.

7.2. Hướng cải tiến

- Thu nhiều lượt có nhãn độc lập để đo nhận đúng, bỏ sót và nhầm lớp theo điều kiện ảnh.
- Chuẩn hóa Wave theo kích thước bàn tay; đánh giá đồng thời confidence, thời gian ổn định và timeout.
- Điều khiển theo độ dịch chuyển có dấu, giảm tốc gần đích; đo tư thế sau khi dừng ổn định.
- Ghi đồng bộ cử chỉ–lệnh–vận tốc để đo trễ; bổ sung cảnh không tự chạy lại sau dừng/mất tín hiệu.

Tài liệu tham khảo

[1] Google AI Edge, *Gesture recognition task guide*. https://developers.google.com/edge/mediapipe/solutions/vision/gesture_recognizer. Truy cập 24/10/2026.

[2] Google AI Edge, *Gesture recognition guide for Python*. https://developers.google.com/edge/mediapipe/solutions/vision/gesture_recognizer/python. Truy cập 24/10/2026.

[3] OpenCV, *Getting Started with Videos*. https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html. Truy cập 24/10/2026.

[4] Open Robotics, *Understanding topics*, ROS 2 Jazzy documentation (bản nguồn). https:

//github.com/ros2/ros2_documentation/blob/jazzy/source/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.rst. Truy cập 24/10/2026.

[5] ROS 2 Common Interfaces, *TwistStamped message*, nhánh Jazzy. https://github.com/ros2/common_interfaces/blob/jazzy/geometry_msgs/msg/TwistStamped.msg. Truy cập 24/10/2026.

[6] TurtleBot 4 User Manual, *TurtleBot 4 Simulator*. https://turtlebot.github.io/turtlebot4-user-manual/software/turtlebot4_simulator.html. Truy cập 24/10/2026.

[7] Gazebo, *Troubleshooting – Harmonic*. https://gazebosim.org/docs/harmonic/troubleshooting/. Truy cập 24/10/2026.

[8] Mã nguồn thực hiện bài thực hành, package `src/turtlebot4_gesture_control`; cấu hình `config/behavior.yaml` và các node được đối chiếu khi viết báo cáo.

[9] Bộ dữ liệu thực nghiệm 24/10/2026: video `demo_turtlebot4_gestures.mp4`, `events.jsonl`, `odometry.jsonl`, `results.csv` và ảnh minh chứng trong thư mục lượt quay.

Phụ lục: Bộ minh chứng thực nghiệm

Bộ minh chứng gồm video MP4 180,05 giây, ảnh webcam và trạng thái, `events.jsonl`, `odometry.jsonl`, `results.csv`. Các file được lưu trong lượt quay ngày 24/10/2026 và gói `lab06_demo_2026-10-24.zip`. Hướng dẫn biên dịch pdfLaTeX và thay thông tin bìa nằm trong README của gói mã nguồn báo cáo.